Real-Time Traffic Sign Detection and Classification

Mannat Varshney | Revised 4 November 2026

Real-time operation is the project goal, not a verified road-video performance claim. Measured speed is reported in Section 3.5.

Abstract

This project develops a five-category traffic-sign detection and classification system using only the Mapillary Traffic Sign Dataset (MTSD). Annotated crops compare ResNet-18, MobileNetV3-Small and EfficientNet-B0; a source-resolution ablation tests preserving original crop detail. Road scenes train YOLO11n. Both tasks inherit the same scene split: 2,018 training, 356 validation and 332 local test scenes. The selected classifier achieves 99.53% on 425 supplied test crops. Detector mAP50 is 0.5475 and mAP50:95 is 0.3734. Crop classification, full-scene detection and processing speed are evaluated separately, including actual ONNX inference.

1. Introduction and Motivation

Traffic-sign inspection is a practical computer-vision task: a road photograph may contain a small, blurred or partly obscured sign among buildings, vehicles and other objects. The objective is to locate supported signs and label them as stop, no entry, yield, speed limit 30 or speed limit 50. The tool saves annotated images or video and a detection table. It is not a driving controller and does not decide whether a road is safe.

The applied problem is finding small signs within complex road photographs. The project tests whether lightweight models and overlapping inference tiles provide useful recognition and localisation under limited computing resources. It saves annotated images/video and prediction tables. It does not claim a complete road-inventory service or measured operational time savings.

The project retains the Lightning/Hydra classifier structure and uses one dataset in two forms: complete photographs for detection and their annotated sign regions for classification. Classification answers which category a supplied rectangle contains; detection must also find that rectangle. MTSD supplies both real scenes and manually annotated boxes [2]. No synthetic training images or second traffic-sign dataset are used in the final experiments.

1.1 Research Questions

The experiments ask: which small classifier performs best under the same training budget; how detector input resolution changes validation detection quality; whether a separately trained crop classifier helps or hurts on detected global road-scene crops; and whether exported ONNX inference preserves the measured detector results. Model selection uses validation data, not final test scores.

1.2 Related Work and Project Contribution

GTSRB and GTSDB motivate the distinction between recognition and detection [1]. The MTSD authors study globally varied signs and explicitly provide full scenes with fine-grained labels [2]. DFG demonstrates traffic-sign inventory as an application [3]. This project uses those ideas for a bounded student implementation: a five-class detector, controlled crop baselines, an executable image/video interface, deployment verification and reproducible experiment records. Published paper scores are not reported as this project's results.

Two additional repositories were reviewed during finalisation [5,6]. The first illustrates webcam detections; the second separates detection from crop recognition. Their class counts, datasets and budgets differ. Their reported scores are not comparable baselines here, and no code or weights were imported. Detailed comparison: report/reference_projects.md.

2. Methodology

2.1 Real Data, Annotations and EDA

The classifiers use 2,563 training crops, 445 validation crops and 425 test crops cut from the prepared MTSD scenes. Crop splits are inherited from source photographs and similarity groups, never assigned independently. Each crop retains source-image identity, rectangle, category and hash. Training crop counts are stop 498, no entry 546, yield 576, speed 30 478 and speed 50 465. Resizing a small real crop does not create new image detail.

The source-resolution ablation preserves these exact crop identities/splits but cuts boxes from original photographs using normalized coordinates. The reviewed speed-50 scene is 5,312 x 2,988 pixels: its 57-59-pixel sign widths become about 14 pixels at 1,280-wide preparation and 7 at 640-wide detector content. Original-source crops avoid that prior downsampling; all classifier inputs are still resized to 128 x 128.

For detection, the authorised MTSD v2 fully annotated release is used. Its annotation ZIP matches the supplied MD5. Original selected image members pass ZIP CRC32 and have recorded SHA256 digests. Range downloads retrieve only the selected images; full image-archive MD5 verification is not claimed. No signs are drawn or pasted into generated scenes.

Category	Train scenes	Val scenes	Test scenes	Train boxes
stop	433	77	71	498
no entry	458	89	71	546
yield	478	84	81	576
speed limit 30	436	65	70	478
speed limit 50	427	73	70	465

One scene can contain several boxes or categories, so category scene counts do not sum to total scenes. More than 400 distinct training scenes per class remain after validation is reserved; this is not 500 independent physical signs per class. Selected official validation scenes form the final local test; hidden test labels are unused. Global speed classes identify the posted number and sign family, not an inferred or converted national speed unit.



Figure 1. Actual MTSD training photograph. Yellow boxes mark only the five selected categories. Large nearby road-bump and oncoming-traffic-priority signs are annotated in the source but outside this task. These are ground-truth boxes, not detector predictions.

2.2 Split Protection and Exclusions

Exact decoded-pixel checks and 64-bit full-scene difference-hash components screen visually similar images. Three training scenes in components overlapping held-out data are removed. Internal training and validation are split by those groups. This screening does not prove physical-sign independence without sequence metadata. Panoramas and scenes with selected target signs flagged ambiguous, occluded or out-of-frame are excluded. The scope is therefore easier than unrestricted driving footage.

Following annotation review, all 2,706 prepared scenes and 3,433 target boxes were compared with the five exact g1 source labels and coordinates: 0 scene mismatches were found. Other source labels are background. This includes 19 related regulatory stop/speed variants and one complementary speed sign not merged in the recorded task. Thus the five names do not imply all international variants are covered. Broader coverage requires reviewed remapping and retraining. The audit verifies mapped-label retention, not completeness of the original annotations.

2.3 Models, Preprocessing and Training

Classifiers use ImageNet initial weights, five MPS epochs, 128-pixel RGB inputs, ImageNet normalisation, AdamW (learning rate 0.0003, weight decay 0.0001), batch 16 and seed 42. Augmentation is rotation/colour jitter, without flips. Validation macro-F1 selects checkpoints/models; CPU evaluates them. The crop-source ablation retains this budget and split.

Initial YOLO11n CPU runs use 416/640 inputs, five epochs, batch 8, AdamW (learning rate 0.001, weight decay 0.0005), seed 42 and ten frozen backbone blocks. Augmentation changes rotation, position, scale and colour, without flips/mosaic/copy-paste. Scenes preserve aspect ratio at up to 1,280 pixels before letterboxing.

The baseline 640 run has five initial epochs plus 15 MPS fine-tuning epochs. The revision adds ten MPS epochs, unfreezing the backbone at learning rate 0.0003. CPU validation mAP50:95 selects baseline or revision before test evaluation; budgets differ, not architectures. Optional classification maps scene boxes back to original photographs for cropping.
SAHI compares whole images with overlapping 640-pixel tiles (20% overlap, whole-image pass, class-aware NMS IoU 0.7). Validation mAP50:95 selects the mode on 356 scenes; validation F1 selects confidence 0.460. Tiles are inference views, not extra independent training images.

2.4 Metrics and Their Selection

Crop accuracy is the fraction correct. Precision measures how often a predicted category is correct; recall measures the fraction of actual members found. F1 = 2PR/(P+R). Macro-F1 weights unequal-support categories equally and selects the classifier. Confusion matrices expose individual category errors.

Detector IoU is box-intersection area divided by union area. mAP50 averages class average precision at IoU >=0.50; mAP50:95 averages ten IoU thresholds, tests tighter localisation and selects detector settings. Unlike crop accuracy, these scores account for misses and false detections. Fixed-threshold P/R/F1 describe one operating point.

3. Experiments and Results

3.1 Mapillary Classifier Comparison

Model	Val F1	Test acc.	Macro P	Macro R	Macro F1
ResNet-18	0.9619	96.94%	0.9695	0.9686	0.9687
MobileNetV3-Small	0.9463	95.76%	0.9564	0.9551	0.9557
EfficientNet-B0	0.9477	96.47%	0.9643	0.9634	0.9633

Validation selects ResNet-18 for the prepared-crop baseline: 412/425 correct test labels. The majority-training-label baseline scores 23.53%. Confusion matrices, predictions, parameters and CPU forward times are saved. These are crop, not scene-detection scores.

ResNet crop source	Val macro-F1	Test accuracy	Test macro-F1
Prepared 1280	0.9619	96.94%	0.9687
Original photograph	0.9927	99.53%	0.9953

Original-source crops are selected by validation. Median crop width rises from 17 to 44 pixels; 423/425 test crops are correct. This is one-seed crop-source evidence, not improved detector recall or a new dataset. The bundled prepared-crop stop example is predicted as no entry by the selected original-source classifier; its stored stop prediction belongs to the earlier baseline. Example execution is not correctness validation.

3.2 Detector Training and Inference

Run	Val mAP50	Val mAP50:95	Phase min
yolo11n_416	0.1640	0.1049	11.5
yolo11n_640	0.2885	0.1943	24.6
yolo11n_640_finetuned	0.3398	0.2302	36.9
unfrozen_640_additional_10_epochs	0.3916	0.2654	29.8

Validation selects checkpoints before testing. Additional phases change initialization/budget; they are not equal-budget architecture comparisons. All runs use YOLO11n and one seed.

Inference mode	Val mAP50	Val mAP50:95	Mean ms/scene
whole	0.3690	0.2548	51.5
sliced	0.5443	0.3820	411.3

Whole/tiled rows share checkpoint, scenes and fixed-square preprocessing; training-validator preprocessing differs. Tiles preserve more sign pixels but cost extra processing.

3.3 Held-out Road-Scene Results

On 332 selected held-out scenes, detector mAP@0.50 is 0.5475 and mAP@0.50:0.95 is 0.3734. Precision and recall at the maximum-F1 curve point are 0.6704 and 0.4743; they are not the same as the fixed-confidence numbers below.

Category	AP50	AP50:95
stop	0.7255	0.5119
no entry	0.4629	0.2708
yield	0.6462	0.4151
speed limit 30	0.3952	0.2930
speed limit 50	0.5079	0.3762

3.4 Detector Labels versus Crop Refinement

Pipeline	Precision	Recall	F1	Matched label acc.	TP / FP / FN
Detector only	0.6067	0.5082	0.5531	80.39%	216 / 140 / 209
Detector + crop model	0.7163	0.6000	0.6530	100.00%	255 / 101 / 170

Detection counts use validation-selected confidence 0.460, IoU 0.50 and same-class one-to-one matching. Matched-label accuracies use the same 255 class-agnostically matched boxes, excluding missed signs and unmatched predictions; detection P/R/F1 count those errors. They are not overall system accuracy or the separate 425 supplied-crop score. Refinement changes labels, not boxes, and cannot recover misses. Detector-only remains the default.



Figure 2. Actual processed held-out road scene. The demo selects early examples by source category, not correctness, and also contains missed or incorrect detections. A gallery video of these real still photographs is not continuous road footage.

3.5 Processing Time

The warm PyTorch-detector-plus-ONNX-classifier pipeline averages 439.10 ms per scene, with median 436.82 ms and 95th percentile 478.79 ms. Timing includes image reading, detector preprocessing/inference/NMS and crop inference; loading models, scoring and output encoding are excluded. Measurements use Apple M1 / macOS / CPU, two PyTorch threads and two classifier ONNX threads. This is not a continuous-video real-time guarantee.

4. Deployment, Reproducibility and Discussion

4.1 ONNX and Optimisation

ONNX is an exported model format, not a third neural architecture. The selected detector is exported with opset 17, fixed batch size 1 and 640-pixel input. ONNX model structure checks pass. Raw outputs on three real test images are compared numerically with PyTorch; the largest absolute difference is 0.000793. Preprocessing and non-maximum suppression are still provided by the application, not embedded in the graph.

Actual ONNX detection on all 332 local test scenes gives mAP50 0.5475, versus PyTorch 0.5475. The new classifier float32 and dense-only dynamic INT8 exports both score 99.53% on 425 crops. CPU forward means are 11.80/11.73 ms (10 warm-ups, 50 repeats), with sizes 44.708/44.705 MB. No convincing speed/compression gain is established. Convolutions stay float32; this is not detector quantisation or pruning.

4.2 Versioning and Execution

The project uses Git for code, DVC for data/model versions, TensorBoard for training curves, Hydra for classifier configuration, and Lightning for the classifier loop. Detection settings are in pipelines/detection/. The image/video command writes annotated media, CSV detections and measured timing. Dataset terms apply separately from code; temporary authorised download links are not committed. Restoring licensed data requires authorised artifact access or regeneration from an authorised download.

Published Mapillary-only commit 8cf89e7 passes Docker build, container tests and two-frame ONNX inference in Actions run 41420610068. Full retraining inside Docker is unverified; later revisions require their own CI evidence. The submission is a fork of Vladimir-125/lightning-hydra-template with its MIT notice and history. No shared DVC remote is configured; full training needs authorised data/cache access.

4.3 Application and Software Verification

The command-line application accepts an image or video, runs whole/tiled detection, optionally classifies accepted crops, and saves annotated media, CSV detections and timing. Scores from the two models are kept separate. The packaged models are hash-checked after fresh extraction and tested on real inputs. Software tests check data handling, training, export and inference utilities; they do not prove predictive reliability. Visual inspection of original images remains necessary because signs are still missed.

4.4 Difficulties and Limits

The main difficulties were acquiring real scenes, protecting splits, preserving labels and finding small signs. Source checks showed nearby omitted signs were outside the selected categories. Original-resolution cropping preserves available classification detail; it cannot recover a sign the detector misses. Tiled inference addresses a different problem: finding small objects within the full scene. Neither resolves every tiny sign or proves real-world reliability.

The study uses selected positive scenes, excludes difficult targets and uses one seed per run. Hashes do not prove physical-sign independence. The protected holdout was inspected in earlier project development, so this is not an external blind evaluation. Unsupported categories and false alarms on sign-free journeys remain unmeasured. The gallery is qualitative evidence, not a moving-video benchmark. More training, difficult scenes, repeated seeds and independent real-road evaluation are needed.

5. Conclusion

One Mapillary dataset supports detection, crop classification and exported inference. The study compares three classifiers, original-source cropping and detector settings. The selected classifier scores 99.53% on 425 annotated crops; local detector mAP50 is 0.5475. These answer different questions. Missed signs remain; autonomous-driving reliability is not established.

References and Evidence

[1] Stallkamp et al., GTSRB, IJCNN 2011; Houben et al., GTSDB, IJCNN 2013 (related work only, not project data). [2] Ertler et al., MTSD, ECCV 2020. mapillary.com/dataset/trafficsign. [3] Tabernik and Skocaj, IEEE T-ITS 2019. vicos.si/resources/dfg/. [4] Akyon et al., SAHI, ICIP 2022. github.com/obss/sahi. [5] github.com/ayushgayakwad/road-sign-recognition. [6] github.com/pedramebd/traffic-sign-detection-recognition. Repositories reviewed 28 October 2026. Evidence: report/mtsd_classifier_results/, mtsd_results/, mtsd_onnx_results/, mtsd_annotation_review/.